1. 8-Puzzle Problem

Aim: To write a Python program to solve the 8-Puzzle Problem using Breadth-First Search (BFS).

Algorithm:

1. Start with the initial puzzle state.
2. Set the goal state.
3. Add the initial state to the queue.
4. Remove a state from the queue and check whether it matches the goal state.
5. If not, move the blank space up, down, left, or right wherever possible.
6. Add the new states to the queue if they have not been visited.
7. Repeat until the goal state is reached.
8. Display the solution path and stop.

Code:

```
from collections import deque
start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)
queue = deque ([(start, [])])
visited = {start}
while queue:
   state, path = queue. Popleft ()
   if state == goal:
      print ("Solution found:")
      for step in path + [state]:
         print (step [:3])
         print (step [3:6])
         print (step [6:])
         print ()
      break
   zero = state. Index (0)
```

```
    row, col = divmod (zero, 3)
    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        r, c = row + dr, col + dc
        if 0 <= r < 3 and 0 <= c < 3:
            new_pos = r * 3 + c
            temp = list(state)
            temp[zero], temp[new_pos] = temp[new_pos], temp[zero]
            new_state = tuple(temp)
            if new_state not in visited:
                visited.add (new_state)
                queue.append ((new_state, path + [state]))
```

Input:

Initial State:

1 2 3

4 0 6

7 5 8

Output:

```
Solution found:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

Result: Thus, the 8-Puzzle Problem was solved successfully using Python.